

- a) Explain the architecture of LSTM and GRU in detail. What were the shortcomings of RNN that were resolved by LSTM and GRU.

Architecture of LSTM and GRU and the shortcomings of RNN

A Recurrent Neural Network (RNN) is a deep learning architecture designed to process sequential data, such as text, speech and time-series information. It maintains a hidden state that carries information from previous time steps to the current time step. However, traditional RNNs suffer from the vanishing-gradient and exploding-gradient problems during backpropagation through time. In the vanishing-gradient problem, gradients become extremely small as they propagate through a long sequence, making it difficult for the network to learn long-term dependencies. In the exploding-gradient problem, gradients become excessively large, leading to unstable training. Moreover, because an RNN continuously updates its hidden state, important information from earlier time steps may gradually be overwritten. To overcome these limitations, advanced recurrent architectures such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) were developed.

Long Short-Term Memory (LSTM) is an advanced type of RNN specifically designed to capture long-term dependencies in sequential data. Unlike a traditional RNN, LSTM maintains two states: a hidden state, which represents the current output, and a cell state, which acts as long-term memory. Its architecture consists of three major gates: the forget gate, input gate and output gate. The forget gate determines which information from the previous cell state should be retained or discarded. The input gate decides how much new information should be stored in the cell state, while a candidate cell state generates potential new information using the current input and previous hidden state. The cell state is then updated by combining the retained old information with the selected new information. Finally, the output gate determines which information from the updated cell state should be passed to the current hidden state. These gates use sigmoid activation functions to regulate information flow, while the candidate memory and hidden-state calculations use the tanh activation function. The cell state's additive update provides a more direct pathway for information and gradients, allowing LSTM to preserve relevant information over longer sequences and substantially reduce the vanishing-gradient problem.

Gated Recurrent Unit (GRU) is another advanced recurrent architecture designed to overcome the limitations of traditional RNNs while providing a simpler alternative to LSTM. Unlike LSTM, GRU does not maintain a separate cell state; instead, it uses a single hidden state to store and transfer information across time steps. Its architecture consists of two major gates: the update gate and the reset gate. The update gate controls how much information from the previous hidden state should be retained and how much new information should be incorporated. The

reset gate determines how much past information should be considered when generating the candidate hidden state. The candidate hidden state is calculated using the current input and the reset-gated previous hidden state. Finally, the GRU combines the previous hidden state and candidate hidden state according to the update gate to produce the current hidden state. This gated mechanism enables GRU to capture long-term dependencies and mitigate the vanishing-gradient problem. Since GRU uses fewer gates and generally has fewer trainable parameters than LSTM with the same hidden size, it is often computationally more efficient and may train faster.

In conclusion, both LSTM and GRU were developed to address the difficulty traditional RNNs face in learning long-term dependencies, particularly because of vanishing gradients and the gradual loss of earlier information. LSTM achieves this through a separate cell state and three gates, whereas GRU uses a simpler architecture with a single hidden state and two gates. Both architectures improve information retention and gradient flow, although neither completely eliminates all gradient-related problems. LSTM provides more explicit control over memory, while GRU offers a computationally simpler alternative that can achieve comparable performance on many sequential learning tasks.

b) Explain the different types of feature engineering techniques like one hot encoding, bag of words, TFIDF etc. How word2vec is different from them. Explain its working with example.

One hot encoding

One hot encoding is a feature engineering technique that converts words into numeric vector such that model can understand it. Suppose we have words cat, fish, dog and car so one hot encoding will convert cat=[1,0,0,0], Fish=[0,1,0,0], dog=[0,0,1,0], car=[0,0,0,1]. Only one position is one hence one hot encoding.

Disadvantages of One hot encoding:-

- 1.If we have 100000 words then vector will become very large and each word gets vector length of 100000
- 2.It doesn't capture relationships between words i.e Euclidean distance between cat and dog is same as of cat and car which square root of 2 but cat and dog are much more similar than cat and car.

Bag of Words

D1-I love dogs

D2-I love cats

D3-Dogs love cats

Vocabulary=[I,love,dogs,cats]

Document	I	love	dogs	cats
D1	1	1	1	0
D2	1	1	0	1
D3	0	1	1	1

This is called document term matrix

Disadvantages

1.Consider dog bites man and man bites dog both will have same vector according to word2vec but in reality both have different meanings

N Grams/Bag of n- grams

Suppose we have following documents

D1-People watch tiger

D2-Tiger watch tiger

D3-Tiger write comment

Vocabulary=["People","Watch","tiger","watch","comment","People watch","watch tiger","Tiger watch","tiger write ","write comment","people watch tiger","tiger watch tiger","tiger write comment"]

People watch tiger=[1,1,1,0,0,1,1,0,0,0,1,0,0,0]

Disadvantages:-

1.Vocabulary size can become very large

2.Sparse matrix

3.High dimensionality

4.More memory usage

TF-IDF

Term frequency inverse documentary frequency

This assigns different weights to each word

$TF(t,d) = \text{No. of occurrences of term } t \text{ in document } d / \text{Total number of terms in document } d$

$IDF(t) = \text{Loge}(\text{Total no. of documents in corpus} / \text{No of documents with term } t \text{ in them})$

D1-People watch tiger

D2-Tiger watch tiger

D3-People write comment

D4-Tiger write comment

$TF(\text{people}, d1) = 1/3$

$IDF(\text{people}) = \text{Loge}(4/2)$

	people		watch		tiger		write		comment
People watch tiger	$(1/3) * \text{loge}(2)$		$(1/3) * \text{loge}(4/2)$		$(1/3) * \text{loge}(4/4)$				

TF-IDF asks is this word important in this document compared with the rest of the corpus

Word2Vec

This is a deep learning based approach.

Word2Vec is a **word embedding technique** used in Natural Language Processing (NLP) to convert words into **dense numerical vectors** that capture relationships between words based on their **context**. Unlike One-Hot Encoding, Bag of Words, or TF-IDF, where words are represented independently based mainly on their identity or frequency, Word2Vec learns the meaning of a word by looking at the words that appear around it. The basic idea is the **distributional hypothesis**: *words that occur in similar contexts tend to have similar meanings*. Word2Vec has two main architectures: **CBOW (Continuous Bag of Words)** and **Skip-gram**. In CBOW, the model predicts a target word from its surrounding words, whereas in Skip-gram, the model predicts surrounding words from a target word.

For example, consider the sentences "**I drink coffee every morning**", "**I drink tea every morning**", and "**I drink juice every morning**." If we use Skip-gram and take "**coffee**" as the target word, the surrounding words such as "**drink**", "**every**", and "**morning**" become its context. Similarly, when the model sees "**tea**" and "**juice**", it observes many of the same surrounding words. During training, Word2Vec starts with randomly initialized weights and repeatedly tries to predict context words. It calculates the prediction error, uses **backpropagation** to calculate gradients, and updates the weights. After seeing a large number of examples, words that occur in similar contexts develop similar vector representations. For example, the model might learn vectors such as coffee = [0.72, 0.31, -0.18, 0.55] and tea = [0.69, 0.35, -0.15, 0.51]. These vectors are called **word embeddings**. Because the vectors for "coffee" and "tea" are similar, their **cosine similarity** will be high, indicating that they have similar contextual meanings. Word2Vec can therefore capture relationships that TF-IDF cannot, such as the similarity between "**king**" and "**queen**", "**car**" and "**automobile**", or "**doctor**" and "**physician**".